

A Data-Driven Prediction Framework for Analyzing and Monitoring Business Process Performances

Antonio Bevacqua¹, Marco Carnuccio¹, Francesco Folino²,
Massimo Guarascio², and Luigi Pontieri²(✉)

¹ DIMES Department, University of Calabria,
via P. Bucci 41C, 87036 Rende, CS, Italy
{abevacqua,mcarnuccio}@deis.unical.it

² ICAR-CNR, National Research Council of Italy,
via P. Bucci 41C, 87036 Rende, CS, Italy
{ffolino,guarascio,pontieri}@icar.cnr.it

Abstract. This paper presents a framework for analyzing and predicting the performances of a business process, based on historical data gathered during its past enactments. The framework hinges on an inductive-learning technique for discovering a special kind of predictive process models, which can support the run-time prediction of a given performance measure (e.g., the remaining processing time/steps) for an ongoing process instance, based on a modular representation of the process, where major performance-relevant variants of it are modeled with different regression models, and discriminated on the basis of context variables. The technique is an original combination of different data mining methods (ranging from pattern mining, to non-parametric regression and predictive clustering) and ad-hoc data transformation mechanisms, allowing for looking at the log traces at a proper level of abstraction, in a pretty automatic and transparent way. The technique has been integrated in a performance monitoring architecture, meant to provide managers and analysts (and possibly the process enactment environment) with continuously updated performance statistics, as well as with the anticipated notification of likely SLA violations. The approach has been validated on a real-life case study, with satisfactory results, in terms of both prediction accuracy and robustness.

Keywords: Data mining · Prediction · Business process management

1 Introduction

In most real application contexts, business processes are bound to the achievement of business goals expressed in terms of performance measures (or Key Performance Indicators), which are monitored continuously at run-time. In principle, historical log data, gathered during past enactments of a process, are a valuable source of hidden information on its behavior, which can be extracted

with the help of process mining techniques [1], and eventually exploited to improve the process, and meet performance-oriented goals. In particular, it appears really relevant, to this regard, the recent research stream on the automated discovery of predictive process models (see, e.g., [3, 7, 9]) for estimating some given performance measure over new instances of a process. The interest towards such novel mining tools stems from the observation that performance forecasts can highly improve process enactments, through, e.g., task/resource recommendations [14] or risk notification [5]. In general, these approaches try to induce some kind of prediction model for the given performance measure, based on a suitable trace abstraction function (mapping, e.g., the trace onto the set/multiset of process tasks appearing in it). For example, a non-parametric regression model is used in [7] to build the prediction for a new (possibly partial) trace, based on its similarity towards a set of historical ones – where the similarity between two traces is evaluated by comparing their respective abstract views. However, such instance-based schemes are likely to be unpractical in real environments, due to their long prediction times. A model-based prediction scheme is conversely followed in [3], where an annotated finite-state machine (AFSM) model is induced from the input log, with the states corresponding to abstract representation of log traces. The discovery of such AFSM models was combined in [9] with a context-driven (predictive) clustering approach, so that different execution scenarios can be discovered for the process, and equipped with distinct local predictors.

A key point in the induction of such models, especially in the case of complex business processes, is the definition of a suitable trace abstraction function, allowing to focus on the properties of log events that impact the more on its performance outcomes. In fact, as discussed in [3], choosing the right abstraction level is a delicate task, requiring to reach an optimal balance between the risks of overfitting (i.e., having an overly detailed model, nearly replicating the training set, which will hardly provide accurate forecasts over unseen cases) and of underfitting (i.e., the model is too abstract and imprecise, both on the training cases and on new ones). In current approaches, the responsibility to tune the abstraction level is left to the analyst, who can select the general form of trace abstractions (e.g., set/lists of tasks), while possibly fixing a maximal horizon threshold h — i.e., only the properties of the h more recent events in a trace may appear in its abstract view. Further, FSM-like models cannot effectively exploit non-structural (context) properties of the process instances — in fact, including such data in trace abstractions would lead to a combinatorial explosion of model states.

Contribution. First of all, we try to overcome the above limitations, by devising a novel approach which can fully exploit “non structural” context data, and find a good level of abstraction over the history of process instances, in a pretty automated and transparent fashion. Our core idea is that handy (and yet accurate enough) prediction models can be learnt through various model-based regression methods (either parametric, such as, e.g., [11, 13], or non-parametric, such as, e.g., [12, 15]), rather than resorting to an explicit representation of process states

(like in [3,9]) or to an instance-based approach (like in [7]). This clearly requires that an adequate propositional representation of the given traces be preliminary build, capturing both structural (i.e., task-related) and (“non-structural”) aspects. To this end, we convert each process trace into a set or a multi-set of process tasks, and let the regression method decide automatically how the basic structural elements of such an abstracted trace view are to be used to make a forecast.

Leveraging the idea of [9], of combining performance prediction with a *predictive clustering* technique, we also try to equip different context-dependent execution scenarios (“variants”) of the analyzed process with separate regression models. In fact, such an approach brings, in general, substantial gain in terms of readability and accuracy, besides explicitly showing the dependence of discovered clusters on context features, and speeding up (and possibly parallelize) the computation of regression models. In fact, as confirmed by the empirical results in Sect. 6, even very simple regression methods could furnish robust and accurate predictions, when combined with an effective clustering procedure. Notably, the target features used in the clustering (where context variables conversely act as descriptive attributes) are derived from frequent structural patterns (still defined as sets or bags of tasks), rather than directly using the abstract representations extracted by the log, as done in [9]. Notably, such an approach frees the analyst from the burden of explicitly setting the abstraction level (i.e., the size of patterns, in our case), which is determined instead in a data-driven way.

Finally, we devise a novel Business Process Analysis architecture, where the predictive performance models obtained with the proposed learning approach are used as a basis for the provision of advanced monitoring and analysis functionalities. In particular, in this architecture, the high-level information captured in the different kinds of process models discovered (i.e., frequent execution patterns, predictive clustering models, and cluster-wise regressors), can be exploited to better comprehend how the real behavior of the process depends on processing steps and on context factors. Moreover, such models can be reused to deploy advanced forecast services, capable to estimate, at run-time, the performance outcomes of new process instances, as well as to notify likely violations of Service Level Agreements, in advance.

Organization. The remainder of the paper is structured as follows. We first introduce some preliminaries (Sect. 2), and formally define a series of key concepts (Sect. 3). The proposed learning algorithm is then illustrated in details in Sect. 4, while Sect. 5 describes the implemented system prototype. After discussing an empirical analysis conducted on a real-life case study in Sect. 6, we finally draw a few concluding remarks and future work directions in Sect. 7.

2 Preliminaries

Log data. As usually done in the literature, we assume that for each process instance (a.k.a “case”) a *trace* is recorded, storing the sequence of *events* happened during its unfolding. Let $\mathcal{T}$ be the universe of all (possibly partial) traces

that may appear in any log of the process under analysis. For any trace $\tau \in \mathcal{T}$, $len(\tau)$ is the number of events in τ , while $\tau[i]$ is the i -th event of τ , for $i = 1 \dots len(\tau)$, with $task(\tau[i])$ and $time(\tau[i])$ denoting the task and timestamp of $\tau[i]$, respectively. We also assume that the first event of each trace is always associated with task A_1 , acting as unique entry point for enacting the process. This comes with no loss of generality, seeing as, should the process not have such a unique initial task, it could be added artificially at the beginning of each trace, and associated with the starting time of the corresponding process instance.

Let us also assume that, like in [9], for any trace τ , a tuple $context(\tau)$ of data is stored in the log to keep information about the execution context of τ , ranging from internal properties of the process instance to environmental factors pertaining the state of the process enactment system. For ease of notation, let $\mathcal{A}^T$ denote the set of all the tasks (a.k.a., activities) that may occur in some trace of $\mathcal{T}$, and $context(\mathcal{T})$ be the space of context vectors — i.e., $\mathcal{A}^T = \cup_{\tau \in \mathcal{T}} tasks(\tau)$, and $context(\mathcal{T}) = \{context(\tau) \mid \tau \in \mathcal{T}\}$.

Further, $\tau[i]$ is the *prefix* (sub-)trace containing the first i events of a trace τ and the same context data (i.e., $context(\tau[i]) = context(\tau)$), for $i = 0 \dots len(\tau)$.

A *log* L is a finite subset of $\mathcal{T}$, while the *prefix set* of L , denoted by $\mathcal{P}(L)$, is the set of all the prefixes of L 's traces, i.e., $\mathcal{P}(L) = \{\tau[i] \mid \tau \in L \text{ and } 1 \leq i \leq len(\tau)\}$.

Let $\hat{\mu} : \mathcal{T} \rightarrow \mathbb{R}$ be an (unknown) function assigning a performance value to any (possibly unfinished) process trace. For the sake of concreteness, we will focus hereinafter on two particular instances of such a function, where the performance measure corresponds to the *remaining time* (denoted by μ_{RT}) or the *remaining steps* (denoted by μ_{RS}), i.e., the time (resp., steps) needed to finish the corresponding process enactment. In general, we assume that performance values are known for all prefix traces in $\mathcal{P}(L)$, for any given log L . This is clearly true for the two measures mentioned above. Indeed, for each trace τ , the (actual) remaining-time of $\tau[i]$ is $\hat{\mu}_{RT}(\tau[i]) = time(\tau[len(\tau)]) - time(\tau[i])$, while the number of remaining steps is $\hat{\mu}_{RS}(\tau[i]) = len(\tau) - i$.

A (predictive) *Process Performance Model (PPM)* is a model that can predict the unknown performance value (i.e., the remaining time/steps in our setting) of a process enactment, based on the contents of the corresponding trace. Such a model can be viewed as a function $\mu : \mathcal{T} \rightarrow \mathbb{R}$ estimating $\hat{\mu}$ all over the trace universe — including the prefix traces of all possible process instances. Learning a PPM hence amounts to solving a particular induction problem, where the training set takes the form of a log L , and the value $\hat{\mu}(\tau)$ of the target measure is known for each (sub-)trace $\tau \in \mathcal{P}(L)$. Current approaches to this problem [3, 7, 9] relies on applying some abstraction functions to process traces, capturing only those facets of the registered events that influence the most process performances, while disregarding minor details.

Trace Abstraction. An abstracted (structural) view of a trace summarizes the tasks executed during the corresponding process enactment. Two simple ways to build such a view consist in regarding the trace as a tasks' set or multiset (a.k.a. bag), as follows.

Definition 1 (Structural Trace Abstraction). Let $\mathcal{T}$ be a trace universe and $A_1, \dots, A_n$ be the tasks in $\mathcal{A}^T$. A *structural (trace-) abstraction function* $struct^{mode} : \mathcal{T} \rightarrow \mathcal{R}_T^{mode}$ is a function mapping each trace $\tau \in \mathcal{T}$ to an *abstract representation* $struct^{mode}(\tau)$, taken from an *abstractions' space* $\mathcal{R}_T^{mode}$. Two concrete instantiations of the above function, denoted by $struct^{bag} : \mathcal{T} \rightarrow \mathbb{N}^n$ (resp., $struct^{set} : \mathcal{T} \rightarrow \{0, 1\}^n$), are defined next, which map each trace $\tau \in \mathcal{T}$ to a bag-based (resp., set-based) representation of its structure: (i) $struct^{bag}(\tau) = \langle count(A_1, \tau), \dots, count(A_n, \tau) \rangle$, where $count(A_i, \tau)$ is the number of times that task A_i occurs in τ ; and (ii) $struct^{set}(\tau) = \langle occ(A_1, \tau), \dots, occ(A_n, \tau) \rangle$, where $occ(A_i, \tau) = 1$ iff $count(A_i, \tau) > 0$, for $i = 1, \dots, n$. $\square$

The two concrete abstraction “modes” (namely, *bag* and *set*) defined above summarize any trace τ into a vector, where each component corresponds to a single process task A_i , and stores either the number of times that A_i appears in the trace τ , or (respectively) a boolean value indicating whether A_i occur in τ or not. Notice that, in principle, we could define abstract trace representations as sets/bags over another property of the events (e.g., the executor, instead of the task executed), or even over a combination of event properties (e.g., the task plus who performed it).

Example 1. Let us consider a real-life case study pertaining a transshipment process, also used for experiments of Sect. 6. Basically, for each container c passing through the harbor, a distinct log trace τ_c is stored, registering all the tasks applied to c , such as: moving c by means of either a straddle-carrier (*MOV*), swapping c with another container (*SHF*), and loading c onto a ship by a shore crane (*OUT*). Let τ be a log trace encoding a sequence $\langle e_1, e_2, e_3 \rangle$ of three events such that $task(e_1) = task(e_2) = MOV$ and $task(e_3) = OUT$. With regard to the abstract trace representations of Def. 1, it is easy to see that $struct^{bag}(\tau) = [2, 0, 1]$, and $struct^{set}(\tau) = [1, 0, 1]$ — where the traces are mapped into a vector space with dimensions $A_1 \equiv MOV$, $A_2 \equiv SHF$, $A_3 \equiv OUT$. $\triangleleft$

The structural abstraction functions in Definition 1 are a subset of those used in previous approaches to the discovery of predictive process models [3, 7, 9]. To be more precise, [7] also considers the possibility to map a trace into a vector of task durations, as well as to combine multiple structural abstractions with data attributes of the traces, while the other two approaches also allow for abstracting a trace into the list of tasks appearing in it (as an alternative to bag/set-oriented abstractions).

3 Formal Framework

Clustering-Based PPMs. The core idea of predictive clustering approaches [4] is that, based on a suitable clustering model, predictions for new instances can be based on the cluster where they are estimated to belong. Two kinds of features are considered for any element z in a given instance space $Z = X \times Y$: *descriptive*

features and *target* features (to be predicted), denoted by $\text{descr}(z) \in X$ and $\text{targ}(z) \in Y$, respectively. Then, a *predictive clustering model* (PCM), for a given training set $L \subseteq Z$, is a function $q : X \rightarrow Y$ of the form $q(x) = p(c(x), x)$, where $c : X \rightarrow \mathbb{N}$ is a partitioning function and $p : \mathbb{N} \times X \rightarrow Y$ is a (possibly multi-target) prediction function. Clearly, whenever there are more than one target features, q encodes a multi-regression model.

Similarly to [9], we here consider a special kind of PPM model, based on a clustering of process traces. The model, described below, is in fact a predictive clustering one, where context data play the role of descriptive attributes, while the target variables are derived by certain performance values of the traces.

Definition 2 (Clustering-Based Performance Prediction Model (CB-PPM)). Let L be a log (over $\mathcal{T}$), with context features $\text{context}(\mathcal{T})$, and $\hat{\mu} : \mathcal{T} \rightarrow \mathbb{R}$ be a performance measure, known for all $\tau \in \mathcal{P}(L)$. Then a *clustering-based performance prediction model* (CB-PPM) for L is a pair $M = \langle c, \langle \mu_1, \dots, \mu_k \rangle \rangle$, encoding the unknown performance function $\hat{\mu}$ through a predictive clustering model (where k is the number of clusters found for L). Specifically, c is a partitioning function, which assigns any (possibly novel) trace to one of the clusters (based on context data), while each μ_i is the PPM of the i -th cluster — i.e., $c : \text{context}(\mathcal{T}) \rightarrow \{1, \dots, k\}$, and $\mu_i : \mathcal{T} \rightarrow \mathbb{R}$, for $i \in \{1, \dots, k\}$. The performance $\hat{\mu}(\tau)$ of any (partial) trace τ is eventually estimated as $\mu_j(\tau)$, where $j = c(\text{context}(\tau))$. $\square$

Notice that each cluster has its own PPM model, encoding how $\hat{\mu}$ depends on the structure (and, possibly, on the context) of a trace, within that cluster. The prediction for each trace is then made with the predictor of the cluster it is assigned to (by function c).

In general, such an articulated kind of PPM can be built by inducing a predictive clustering model and multiple PPMs (as the building blocks implementing c and all μ_i , respectively). In particular, in [9], the latter task is accomplished by using the method in [3], so that each cluster is eventually provided with an AFSM model. As mentioned in Sect. 1, in order to develop an easier-to-use and data-adaptive approach, we will not use AFSM models (which typically require a careful explicit setting of the abstraction level), and will rather employ one of the various regression methods available for propositional data. To this purpose, an ad-hoc view of the log needs to be produced, where both the context-oriented and structure-oriented (cf. Definition 1) features of a trace are used as descriptive attributes, whereas the target attributes are derived by projecting the trace onto a space of structural patterns. These patterns, eventually computed with an ad-hoc data mining method, are described in detail in the following.

Structural Patterns. In our setting, structural patterns are meant to capture regularities in the structure of log traces, and they correspond to (constrained) sub-sets or sub-bag of tasks appearing frequently in the log traces (viewed as well as tasks' sets/bags). More precisely, let $\text{mode} \in \{\text{bag}, \text{set}\}$ be a given abstraction criterion, $\mathcal{T}$ be the reference trace universe, and $\mathcal{A}^T = \{A_1, \dots, A_n\}$ be its associated process tasks. Then, a (*structural*) *pattern* w.r.t. $\mathcal{T}$ and mode simply

is an element p of the *abstractions' space* $\mathcal{R}_T^{mode}$ – over which function $struct^{mode}$ ranges indeed. The size of p , denoted by $size(p)$, is the number of distinct tasks in p , i.e., the number of p 's components with a positive value.

Having a structural pattern the same form as a (structural) trace abstraction, we can apply usual set/bag containment operators to them all. Specifically, given two elements p and p' of $\mathcal{R}_T^{mode}$ (be them patterns or representations of entire traces), we say that p_2 *contains* p_1 (and that p_1 is contained in p_2), denoted by $p_1 \preceq p_2$, if $p_1[j] \leq p_2[j]$, for $j = 1, \dots, n$, and for $i = 1, 2$ — where $p_i[j]$ is the j -th component of vector p_i , which ranges over $\{0, 1\}^n$ or $\mathbb{N}^n$ depending on the chosen abstraction mode (cf. Definition 1).

Since such patterns are to be eventually used for clustering purposes, we are interested in those capturing significant behavioral schemes. In particular, an important requirement is that they occur frequently in the given log (otherwise, little, low significant clusters will be found), as specified in the following notion of support.

Definition 3 (Pattern Support and Footprint). Let $\tau \in \mathcal{T}$ be a trace, *mode* be a given abstraction mode, and p be pattern (w.r.t. $\mathcal{T}$ and *mode*). Then, τ *supports* p , denoted by $\tau \vdash p$ if its corresponding structural abstraction contains p (i.e., $p \preceq struct^{mode}(\tau)$). In this case, a *footprint* of p on τ is subset $F = \{f_1, \dots, f_k\} \subseteq \{1, \dots, len(\tau)\}$ of positions in τ , such that $struct^{mode}(\langle \tau[f_1], \dots, \tau[f_k] \rangle) = p$. Moreover, $gap(F)$ denotes the number of events in τ which correspond to none of the positions in F but appear in between a pair of matching events, i.e., $gap(F) = \max_{f_i \in F} \{f_i\} - \min_{f_i \in F} \{f_i\} - |F| + 1$. We finally denote $gap(p, \tau) = \min \{ \{\infty\} \cup \{ gap(F) \mid F \text{ is a footprint of } p \text{ on } \tau \} \}$.

□

A footprint F of a pattern p , on a trace τ supporting it, identifies a subsequence of τ which contains all the elements of τ marked by F (i.e., appearing in one of the positions of F). Clearly, the structural representation of such a subsequence coincides with p . As a special case, if τ does not support p , then $gap(p, \tau)$ is infinite.

In order to focus on significant patterns, w.r.t. a given log L , one could simply set a minimal support threshold, say $minSupp \in [0, 1)$, and discard all patterns getting a support lower than $minSupp \times |L|$ by the given log traces (viewed as tasks' bags or sets). In addition, an upper threshold $maxGap \in \mathbb{N} \cup \{\infty\}$ can be specified for the maximal gap admitted between patterns and traces, in order to focus on patterns that fit well enough the actual sequencing of tasks in the latters. Both constraints can be specified through a variant of the classical support function (actually coinciding with the latter when $maxGap = \infty$), defined as follows:

$$supp^{maxGap}(p, L) = \frac{|\{\tau \in L \mid \tau \vdash p \text{ and } gap(p, \tau) < maxGap + 1\}|}{|L|} \quad (1)$$

Let L be a log, p be a structural pattern for a given abstraction mode (*BAG* or *SET*), $minSupp$ be a minimum support threshold, and $maxGap$ be a maximum gap threshold. Then, p is a $(minSupp, maxGap)$ -frequent pattern w.r.t. L

Input:	A log L over a trace universe $\mathcal{T}$, with associated tasks $AS = A_1, \dots, A_n$ and target performance measure $\hat{\mu}$ (known over $\mathcal{P}(L)$), an abstraction mode $m \in \{set, bag\}$ (cf. Def. 1), three thresholds, $minSupp \in [0, 1)$, $maxGap \in \mathbb{N} \cup \{\infty\}$, and $K_{top} \in \mathbb{N}^+ \cup \{\infty\}$, and a base regression method $REGR$.
Output:	A CB-PPM model for L (fully encoding $\hat{\mu}$ all over $\mathcal{T}$).
Method:	Perform the following steps:
1	Let $context(\tau)$ be the vector of context data associated with each $\tau \in L$;
2	Build a <i>structural view</i> S_L of $\mathcal{P}(L)$, by replacing each $\tau \in \mathcal{P}(L)$ with a transaction-like representation of $struct^m(\tau)$;
3	$RSP := minePatterns(S_L, m, minSupp, maxGap)$;
4	$RSP := filterPatterns(RSP, K_{top})$;
6	Let $RSP = \{p_1, \dots, p_s\}$;
7	Build a <i>log sketch</i> P_L for L , by using both context data and RSP -projected performances;
8	Learn a PCT T , using $context(\tau)$ (resp., $val(\tau, p_i)$, $i=1..s$) as descriptive (resp., target) features for each $\tau \in L$;
9	Let $L[1], \dots, L[k]$ denote the discovered clusters;
10	for each $L[i]$ do
11	Induce a regression model ppm_i out of $\mathcal{P}(L[i])$, using method $REGR$ — regarding, for each $\tau \in \mathcal{P}(L[i])$, $context(\tau)$ and $struct^m(\tau)$ as the input values, and the performance measurement $\hat{\mu}(\tau)$ as the target value;
10	Store ppm_i as the implementation of the prediction function $\mu_i : \mathcal{T} \rightarrow M$ (for cluster i);
11	end
12	return $\langle c, \{\mu_1, \dots, \mu_k\} \rangle$.

Fig. 1. Algorithm AA-PPM Discovery.

if $supp^{maxGap}(p, L) \geq minSupp$. In our approach, such patterns are considered as interesting behavioral features, useful for finding a performance-relevant partitioning of the log traces.

4 Learning Algorithm

Figure 1 illustrates the main steps of our approach to the discovery of a CB-PPM model, in the form of an algorithm, named **AA-PPM Discovery**. Essentially, the problem is approached in three main phases.

In the first phase (Steps 1–5), a set of (frequent) structural patterns are extracted from the log, which are deemed to capture the main behavioral schemes of the process, as concerns the dependence of performance on the execution of tasks. To this end, after converting the structure of each (possibly partial) trace τ into an itemset (Step 2), we compute all the structural patterns (i.e., subsets, of various sizes) that occur frequently in the log and effectively summarize the behaviors in the log — in particular, in the case of bag abstractions, notice that any $s = struct^{bag}(\tau) \in \mathbb{N}^n$ can be represented as $\{(A_i, k_j) \mid 0 \leq i \leq n, s[i] > 0 \text{ and } 1 \leq j \leq s[i]\}$.

More precisely, we first compute the set $\{p \in \mathcal{R}_T^m \mid supp^{maxGap}(p, S_L) \geq minSupp\}$ (cf. Definition 3), by using function `minePatterns`, which is stored in RSP — note that this set will never be empty, since (as an extreme case) at least a singleton pattern with A_1 is frequent (no matter of $minSupp$, m and

maxGap). These patterns are then filtered by function `filterPatterns`, which selects the *kTop* most relevant patterns among them. Notably, we can still use all the discovered patterns, by fixing *maxGap* = ∞ (no real filter is applied in this case). Both these functions are explained in details later on.

In the second phase, the selected patterns are used to associate a series of numerical variables with all traces (Step 7), and to carry out a predictive clustering of them (Step 8). To this end, a propositional view of the log, here named *log sketch*, is produced by transforming each trace into a tuple, where context properties play as descriptive attributes and the projection onto the space of selected patterns are the target numerical features. Specifically, any selected pattern *p* gives rise to a target (performance) feature, such that the value *val*(τ, p) taken by it on any trace τ is computed as follows: (i) *val*(τ, p) = NULL, if $\tau \not\models p$, or (ii) *val*(τ, p) = $\hat{\mu}(\tau(j^*))$, where j^* is the biggest index $j \in \{1, \dots, \text{len}(\tau)\}$ such that $\tau(j) \vdash p$. Like in [9], the clustering is computed by inducing a Predictive Clustering Tree (PCT) [4] from the log sketch (Step 8).

Finally, each cluster is equipped with a basic (not clustering-based) PPM model, by using some suitable regression method (chosen through parameter *REGR*), provided with a dataset encoding all the prefixes that can be derived from the traces assigned to the cluster. Specifically, each such prefix τ is encoded as a tuple where *context*(τ) and *struct*^m(τ) are regarded as input values, while the associated performance measurement $\hat{\mu}(\tau)$ represents the value of the numerical target variable that is to be estimated.

Function *minePatterns*. This function is devoted to compute a set of (*minSupp*, *maxGap*)-frequent patterns of any size — i.e., patterns getting a support score (according to Eq. 1) equal to or higher than *minSupp*, over a transaction-oriented view of the log. Notably, the function does not require the analyst to specify the size of each pattern (differently from the horizon threshold *h* of previous methods), which is actually chosen automatically, in a data-driven way. However, it allows for possibly fixing a finite *maxGap* threshold for the gaps admitted between patterns and traces, in case she/he wants to keep more details on the actual tasks' sequencing. All the specified constraints are enforced along the computation, in order to shrink the amount of patterns generated, as well as the computation time. Further details are omitted for lack of space.

Function *filterPatterns*. This function is meant to select a subset of significant and useful patterns, as to make the computation of clusters more effective and more scalable, by preventing the PCT learning algorithm to work with a sparse and high-dimensional target space. Hence, we allow the analyst to ask for only keeping the *kTop* patterns that seems to discriminate the main performance profiles at best. To this end, we employ a variant of the scoring function ϕ proposed in [9] (giving score 0 to every feature not positively correlated with context data), which gives preference to patterns ensuring higher values of the following measures: (i) support, (ii) correlation with the context attributes and (iii) and variability of the associated performance values (i.e., *val*(p, τ), with τ ranging over *L*). Further details can be found in [9].

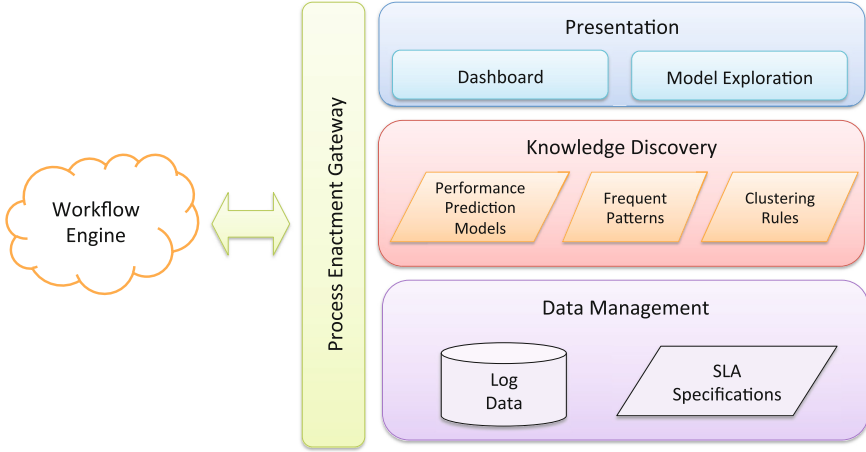


Fig. 2. Conceptual architecture of the *AA-TP* system prototype.

5 System AA-TP (Adaptive-Abstraction Time Prediction)

The learning approach described so far has been integrated into a prototype system, aimed at providing advanced services for the analysis and “anticipated” (i.e., predictive) monitoring of process performances. The current implementation of the system focuses on the two time-oriented process performance measures considered in this paper: the remaining processing time (μ_{RT}), and the number of remaining processing steps (μ_{RS}).

Looking at the right part of Fig. 2, it can be seen that the system features a three-layer conceptual architecture. The lowest level implements basic *Data Management* modules, which are responsible for storing both historical process logs and different kinds of data (e.g., structural log views and log sketches), derived from them prior to the application of the inductive learning mechanisms described in the previous section. In fact, such data, originated by past enactments of a business process, provide a basis for the automated discovery of three different kinds of knowledge about the process: (i) frequent (structural) execution patterns, (ii) predictive clustering models (capturing the existence of diverse execution scenarios for the process, as well as their correlation with major context factors), and (iii) a series of *PPM* models (one for each discovered scenario). All these pieces of knowledge, built and handled in the *Knowledge Discovery* layer, can help better comprehend and analyze the real behavior of the process, and the dependence of its performances on both processing steps and facets of the execution context. On the other hand, thanks to their predictive nature, each clustering model and its associated local *PPMs* can be reused to configure a forecasting service for the process they were discovered from, in order to estimate (at run time and step-by-step) the performance outcome of any new instance of that process. Essentially, based on a given performance prediction model, the service implements a core method, which takes as input the partial

trace of an ongoing process enactment (encoded in one of the standard formats used in ProM [2]), and returns an estimate for the associated measure (i.e., the remaining processing time/steps). Further details on the implementation of knowledge discovery functionalities are described in the final part of this section.

In principle, each deployed forecasting service could be accessed, through the *Process Enactment Gateway* interface module, by any workflow engine or any other kind of enactment system, in order to improve the process at run-time, based on ad-hoc optimization policies. The same module also offers basic services for entering new log data, and for updating a performance prediction model (based on newly added data).

In addition to pure performance forecasting services, the system supports the anticipated notification of Service Level Agreement (SLA) violations, whenever a process instance is deemed as very likely to eventually fail a given quality requirement, previously established for the corresponding process (and some of its associated performance measures). The provision of such a higher-level prediction service requires that an SLA model can be defined for any process P and performance measure μ , and be kept in a suitable repository of the Data Management layer. Basically, such a model is meant to encode information about: (i) the procedure for computing the value of μ for any historical (completely executed) instance of P ; (ii) a violation criterion, stating which are values that μ is not admitted to take on P 's instances; (iii) the degree of sensitiveness in the notification of possible violations. A simple approach to the specification of SLA models and associated triggering mechanisms is explained later on.

Finally, the *Presentation Layer* (still at an incomplete stage of development) is in charge of providing the user (analyst or manager) with summarized information on the past and current behavior of the analyzed business processes. In particular, it allows the user to inspect and navigate all kinds of process models and patterns extracted out of log data, as well as to create and access SLA specifications. Moreover, a customizable dashboard can be build to provide the user with aggregated statistics computed over the instances of a given process or on predefined groups of them (e.g., cases enacted for the same customer), possibly mixing predicted performance outcomes with historical ones.

SLA Models. Currently the system allows for specifying a simple kind of SLA model over two types of performance measures: the total processing time and the total number of processing steps. In particular, for each chosen process and performance measure, it is possible to set a maximum threshold M (identifying the legal range of values for the measure), and a risk tolerance threshold γ — the greater the threshold, the lower the sensitivity in detecting SLA violations (w.r.t. to the performances of the process).

Let $\tau[i]$ denote a trace, encoding the history of an ongoing process instance up to the (current) execution step i . Then, an SLA violation for $\tau[i]$ can be predicted on the basis of the likelihood $\ell_M(\tau[i])$ that the total time (or number of steps) needed to fully handle the case corresponding to τ will not exceed M (like in [5]). In more details, letting $elapsed(\tau[i])$ denote the time already spent (resp. steps already performed), this likelihood is computed as follows:

$$\ell_M(\tau) = \begin{cases} 1 - \frac{M}{\text{elapsed}(\tau(i)) + \mu(\tau(i))} & \text{if } \text{elapsed}(\tau(i)) + \mu(\tau(i)) > M \\ 0 & \text{otherwise} \end{cases}$$

where $\mu(\tau(i))$ is the remaining processing time (resp., nr. of steps) estimated for $\tau(i)$.

Then, an alert can be eventually triggered for the process instance associated with $\tau(i)$, if $\ell_M(\tau(i)) > \gamma$. In such a case, the user (or even the enactment system) will get aware of the fact that an SLA is likely to be infringed, so that some suitable optimization policy could be proactively put in place, in order to possibly prevent the occurrence of such a requirement violation.

Details on the Knowledge Discovery Layer. The core learning mechanism in this layer have been implemented as a plugin of the framework ProM [2]. As mentioned above, the plugin specializes algorithm **AA-PPM Discovery** to the case where the target performance measure coincides with the remaining processing time/steps. The plugin features the following major modules: (a) *Scenario Discovery* module, which is in charge of identifying behaviorally homogeneous groups of traces, in terms of both context data and remaining times; and (b) *Time Predictors Learning* module, which implements a range of classical regression algorithms (including, in particular, IB-k, Linear Regression, and RepTree), eventually used to induce the local predictor (i.e., PPM) of each discovered cluster — all of these predictors will compose (along with the logical rules discriminating among the clusters) the overall CB-PPM model, returned as main result.

More specifically, the former module consists of the following submodules: (i) the *Predictive Clustering* submodule, which groups traces sharing similar descriptive and target values, leveraging system CLUS [6] (a framework for inducing PCT models from propositional data); (ii) the *Log-View Generator* submodule, which right converts all log traces into propositional tuples, according to the ARFF format used in CLUS, relying on the explicit representation of both context data and target attributes (derived from the original log); and (iii) the *Pattern Mining* module, which exploits a transactional representation of these context-enriched traces in order to extract a set of relevant patterns out of them. In fact, these patterns are the target features used by the *Log-View Generator* in order to build a training set for the induction of a clustering model.

6 Case Study

This section illustrates a series of experimental activities that we conducted, with the prototype system **AA-TP**, on some logs of a real transshipment system (already mentioned in Example 1), keeping trace of major logistic activities applied a sample of 5336 containers, which passed through the system in the first third of year 2006. Essentially, each container is unloaded from a ship and temporarily placed near to the dock, until it is carried to some suitable yard slot for being stocked. Symmetrically, at boarding time, the container is first placed in a yard area close to the dock, and then loaded on a cargo. Different kinds of vehicles can be used for moving a container, including, e.g., cranes, “straddle-carriers”,

and “multi-trailers”. This basic life cycle may be extended with additional transfers, devoted to make the container approach its final embark point or to leave room for other ones. Several data attributes are available for each container as context data (of the corresponding process instance), including: the origin and final ports, its previous and next calls, various properties of the ship unloading it, physical features (such as, e.g., size, weight), and some information about its contents. Like in [9], we also considered a few more (environment-oriented) context features for each container: the hour (resp., day of the week, month) when it arrived, and the total number of containers that were in the port at that moment.

Considering the remaining processing time/steps as the target performance measure, we evaluated the effectiveness of predictions via three classic error metrics (computed via 10-fold cross validation): *root mean squared error* (*rmse*), *mean absolute error* (*mae*), and *mean absolute percentage error* (*mape*). For ease of interpretation, the results reported next for the former two metrics have been normalized w.r.t. the average processing time/steps (computed over all the containers passed through the terminal).

Tuning of Parameters. First of all, we tried our approach with different settings of its parameters, including the base regression method (*REGR*) for inducing the PPM of each discovered cluster. For the sake of simplicity, we here only focus on the usage of two basic regression methods: classic *Linear* regression [8], and the tree-based regression algorithm *RepTree* [15]. In addition, we consider the case where each PPM model simply encodes a *k*-NN regression procedure (denoted by *IB-k* hereinafter), as a rough term of comparison with the family of instance-based regression methods (including, in particular, the approach in [7]). For all of the above regression methods, we reused the implementations available in the popular data-mining library Weka [10].

Figure 3 allows for analyzing how the *rmse* scores¹ vary when using different regression methods (distinct curves are depicted for them), and different values of the parameters (namely, $maxGap \in \{0, 4, 8, \infty\}$, $kTop \in \{4, \infty\}$, and $minSupp \in \{0.1, \dots, 0.4\}$).

Clearly, the underlying regression method is the factor exhibiting a stronger impact on precision results. In particular, the disadvantage of using linear regression is neat (no matter of the error metrics), whereas both *IB-k* and *RepTree* methods performs quite well, and very similarly. This is good news, especially for the *RepTree* method, which is to be preferred to *IB-k* for scalability reasons. Indeed, this latter may end up being too time-consuming at run-time, when a large set of example traces must be kept – even though, differently from pure instance-based methods (like [7]), we do not need to search across the whole log, but only within a single cluster (selected via context data).

As to the remaining parameters, we notice that poorer results are obtained when $minSupp = 0.1$ and $kTop = 4$, as well as when $minSupp = 0.4$. As a matter of fact, the former case epitomizes the cases where we cut little (according to frequency) during the generation of patterns, while trying to reduce their

¹ Notice that similar trends of behavior were discovered for the *mae* and *mape* metrics.

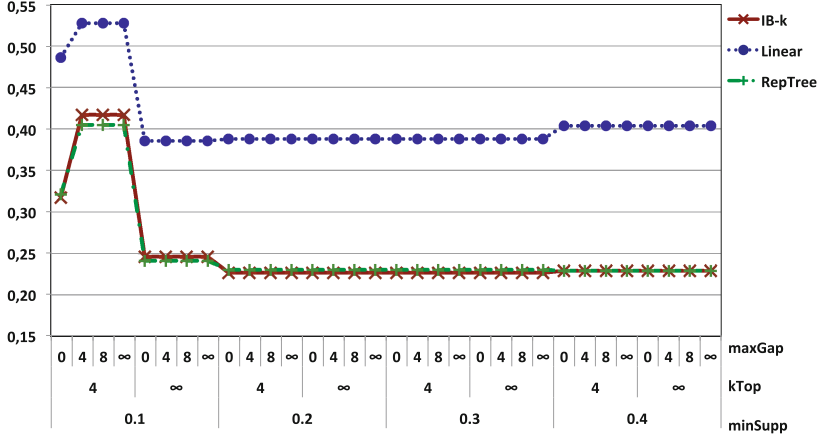


Fig. 3. The effect of parameters on **rmse** results.

number in the filtering phase; the latter, instead, is an opposite situation where a rather high threshold support threshold is employed, at a higher risk of losing important pieces of information on process behaviour. Remarkably, when *minSupp* gets a value from $[0.2, 0.3]$, the remaining two parameters (namely *kTop* and *maxGap*) do not seem to affect the quality of predictions at all. In practice, it suffices to choose carefully the regression method (and a middling value of *minSupp*) to ensure good and stable prediction outcomes, no matter of the other parameters – which would be, indeed, quite harder to tune in general.

Comparison with Competitors. In order to assess the effectiveness of our approach, we compared it with two other ones, defined in the literature for the discovery of a PPM: CA-TP [9] and AFSM [3]. Tables 1 and 2 report the average errors and standard deviations made by system AA-TP, while varying the base regression method (namely, and *IB-k* and *RepTree*), when predicting both remaining times and remaining steps, respectively. In particular, the first half of tables regards the case when *bag* representations are used for abstracting traces, while the second to the usage of *set* abstractions. These values were computed by averaging the ones obtained with different settings of the parameters *minSupp*, *kTop*, and *maxGap*. Similarly, for both of the approaches CA-TP and AFSM, we computed the average of the results obtained using different values of the history horizon parameter *h* (precisely, $h = 1, 2, 4, 8, 16$), and the best-performing setting for all the remaining parameters.

Interestingly, the figures in Tables 1 and 2 indicate that our approach is more accurate than both competitors, no matter which abstraction strategy is adopted. It is worth noticing that the best results (shown in bold in the tables), for all the error metrics, are obtained when AA-TP is used with the *bag* abstraction mode. In particular, by looking at values in Table 1 it is easy to notice that, if we combine this abstraction mode with the *IB-k* regressor, AA-TP manages to lower the prediction error by about 65.88 % on average w.r.t. CA-TP, and

Table 1. Errors (avg $\pm$ stdDev) made by AA-TP and its competitors in predicting remaining times, with different abstraction modes (namely, *bag* and *set*).

Metric	BAG				SET			
	AA-TP (IB-k)	AA-TP (RepTree)	CA-TP	AFSM	AA-TP (IB-k)	AA-TP (RepTree)	CA-TP	AFSM
rmse	0.205 $\pm$ 0.125	0.203$\pm$0.082	0.291 $\pm$ 0.121	0.505 $\pm$ 0.059	0.287 $\pm$ 0.123	0.286 $\pm$ 0.084	0.750 $\pm$ 0.120	0.752 $\pm$ 0.037
mae	0.064$\pm$0.058	0.073 $\pm$ 0.033	0.142 $\pm$ 0.071	0.259 $\pm$ 0.008	0.105 $\pm$ 0.061	0.112 $\pm$ 0.035	0.447 $\pm$ 0.077	0.475 $\pm$ 0.009
mape	0.119$\pm$0.142	0.189 $\pm$ 0.136	0.704 $\pm$ 0.302	0.961 $\pm$ 0.040	0.227 $\pm$ 0.131	0.267 $\pm$ 0.060	2.816 $\pm$ 0.303	2.892 $\pm$ 0.206

Table 2. Errors (avg $\pm$ stdDev) made by AA-TP and its competitors in predicting remaining steps, with different abstraction modes (namely, *bag* and *set*).

Metric	BAG				SET			
	AA-TP (IB-k)	AA-TP (RepTree)	CA-TP	AFSM	AA-TP (IB-k)	AA-TP (RepTree)	CA-TP	AFSM
rmse	0.192$\pm$0.014	0.209 $\pm$ 0.010	0.241 $\pm$ 0.042	0.278 $\pm$ 0.022	0.193 $\pm$ 0.013	0.211 $\pm$ 0.018	0.286 $\pm$ 0.056	0.322 $\pm$ 0.016
mae	0.082$\pm$0.004	0.102 $\pm$ 0.007	0.127 $\pm$ 0.039	0.151 $\pm$ 0.021	0.082 $\pm$ 0.008	0.103 $\pm$ 0.028	0.172 $\pm$ 0.051	0.201 $\pm$ 0.013
mape	0.035$\pm$0.003	0.042 $\pm$ 0.003	0.060 $\pm$ 0.025	0.056 $\pm$ 0.016	0.035 $\pm$ 0.003	0.043 $\pm$ 0.030	0.094 $\pm$ 0.036	0.110 $\pm$ 0.008

by an astonishing 77.51 % w.r.t. AFSM, on average (w.r.t. all the error metrics). Again, good results are obtained when using *RepTree* (still with bag abstractions), where a reduction of 59.10 % (resp., 73.04 %) is achieved w.r.t. to CA-TP (resp., AFSM). An even higher degree of improvement is achieved when using set-oriented abstractions. Indeed, in this case, the reduction in the average of prediction errors is of 84.58 % w.r.t. CA-TP and of 84.97 % w.r.t. AFSM, when using our approach with the *IB-k* regressor; moreover, when using it with *RepTree*, these reductions become 83.43 % and 83.86 %, respectively.

As to the prediction of remaining steps, the best outcomes are obtained when using AA-TP with the *IB-k* regressor (see Table 2). Improvements are, in general, less noticeable than those observed for the prediction of remaining times, but still substantial. Indeed, AA-TP manages to shrink the prediction error of 26.89 % (resp. 16.49 %) w.r.t. CA-TP and of 36.70 % (resp. 27.68 %) w.r.t. AFSM, when using the *IB-k* (resp. *RepTree*) regressor and bag abstractions. Finally, in the case of set abstractions, an error reduction of 43.77 % (resp. 35.39 %) w.r.t. CA-TP, and of 51.04 % (resp. 43.74 %) w.r.t. AFSM is observed when AA-TP is used in combination with *IB-k* (resp., *RepTree*) regressors.

SLA Violation Results. We next focus on a specific kind of SLA violations, corresponding to the overcoming of maximal threshold M on the total processing time. In the context of container management systems, such a maximum processing (“dwell”) time is typically set within predefined agreements, on service quality, between the shipping lines and the terminal handler. In our tests we considered a parametric specification of this threshold: $M = \alpha \times ADT$, where ADT is the average dwell time computed over all containers, and α is an integer ranging over $\{1, 2, 3\}$ (allowing us to simulate three different settings of the agreement level). It is important to notice that cases requiring an excessively long processing time imply high monetary costs, and detecting them in advance

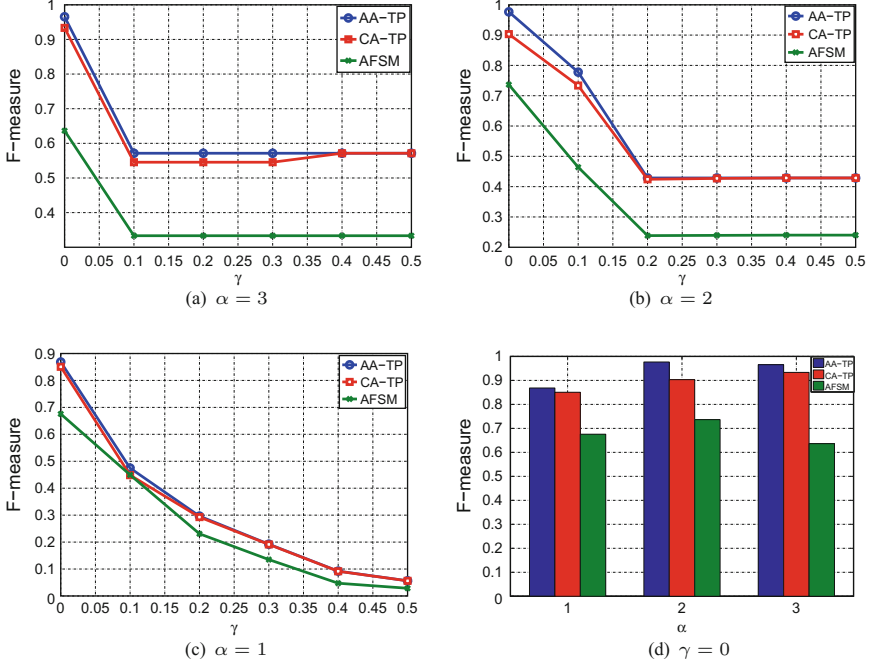


Fig. 4. F-measure scores for the prediction of overtime faults by AA-TP and by baseline methods, when varying γ and the factor α .

can allow for undertaking suitable counter-measures, possibly exploiting additional (storage/processing) resources, which are not used in normal conditions. Since, however, such a remedial policy as well come with a cost, even though it is typically lower than SLA-violation penalties, such an overtime prediction mechanism must exhibit a good trade-off between precision and recall, in order to ensure the whole approach to be really convenient economically.

Figure 4 sheds light on the ability our approach to discriminate “overtime” from “in-time” containers. To this purpose, we report the F-measure scores for different values of the risk threshold γ , when a fixed, good-working, configuration is used for both our approach and the baseline ones, respectively CA-TP [9] and AFSM [3]. As expected, F-measure tends to worsen when increasing γ , whatever the factor α is. Interestingly, when using lower values of γ (i.e., a more aggressive warning policy) the capability of our approach to recognize real overtime cases is ever better than the baseline predictors – in particular, a more evident improvement is obtained for $\alpha = 2$ when an astonishing F-measure of 0.98 is reached w.r.t. 0.90 and 0.74 reached by CA-TP and AFSM, respectively.

7 Conclusions

We have presented a new predictive process-mining approach, which fully exploits context information, and manages to find the right level of abstraction on log traces in data-driven way. Combining several data mining and data transformation methods, the approach allows for recognizing different context-dependent process variants, while equipping each of them with a separate regression model.

Encouraging results obtained on a real application scenario show that the method is precise and robust enough, yet requiring little human intervention. Indeed, it suffices not to use extreme values for the support threshold to have low prediction errors, no matter of the other finer-grain parameters (i.e., *maxGap* and *kTop*).

The technique has been integrated in a performance monitoring architecture, capable to provide managers and analysts with continuously updated performance statistics, as well as with the anticipated notification of possible SLA violations, which could be possibly prevented via suitable improvement policies.

As future work, we plan to explore the usage of sequence-like patterns (e.g., *k*-order subsequences) — possibly combined with those considered here — in order to capture the structure of a process instance in a more precise (but still abstract enough) manner, as well as to integrate our approach with a real BPM environment.

References

1. van der Aalst, W.M.P., van Dongen, B.F., Herbst, J., Maruster, L., Schimm, G., Weijters, A.J.M.M.: Workflow mining: a survey of issues and approaches. *Data Knowl. Eng.* **47**(2), 237–267 (2003)
2. van der Aalst, W.M.P., et al.: ProM 4.0: comprehensive support for *Real* process analysis. In: Kleijn, J., Yakovlev, A. (eds.) ICATPN 2007. LNCS, vol. 4546, pp. 484–494. Springer, Heidelberg (2007)
3. van der Aalst, W.M.P., Schonenberg, M.H., Song, M.: Time prediction based on process mining. *Inf. Syst.* **36**(2), 450–475 (2011)
4. Blockeel, H., Raedt, L.D.: Top-down induction of first-order logical decision trees. *Artif. Intell.* **101**(1–2), 285–297 (1998)
5. Conforti, R., Fortino, G., La Rosa, M., ter Hofstede, A.H.M.: History-aware, real-time risk detection in business processes. In: Meersman, R., et al. (eds.) OTM 2011, Part I. LNCS, vol. 7044, pp. 100–118. Springer, Heidelberg (2011)
6. DLAI Group: CLUS: a predictive clustering system (1998). <http://dtai.cs.kuleuven.be/clus/>
7. van Dongen, B.F., Crooy, R.A., van der Aalst, W.M.P.: Cycle time prediction: when will this case finally be finished? In: Meersman, R., Tari, Z. (eds.) OTM 2008, Part I. LNCS, vol. 5331, pp. 319–336. Springer, Heidelberg (2008)
8. Draper, N.R., Smith, H.: Applied Regression Analysis. Wiley Series in Probability and Statistics. Wiley, New York (1998)
9. Folino, F., Guarascio, M., Pontieri, L.: Discovering context-aware models for predicting business process performances. In: Meersman, R., et al. (eds.) OTM 2012, Part I. LNCS, vol. 7565, pp. 287–304. Springer, Heidelberg (2012)

10. Frank, E., Hall, M.A., Holmes, G., Kirkby, R., Pfahringer, B.: Weka - a machine learning workbench for data mining. In: Maimon, O., Rokach, L. (eds.) *The Data Mining and Knowledge Discovery Handbook*, pp. 1305–1314. Springer, New York (2005)
11. Hardle, W., Mammen, E.: Comparing nonparametric versus parametric regression fits. *Ann. Stat.* **21**(4), 1926–1947 (1993)
12. Harlde, W.: *Applied NonParametric Regression*. Cambridge University Press, Boston (1990)
13. Quinlan, R.J.: Learning with continuous classes. In: *Proceedings of 5th Australian Joint Conference on Artificial Intelligence (AI'92)*, pp. 343–348 (1992)
14. Schonenberg, H., Weber, B., van Dongen, B.F., van der Aalst, W.M.P.: Supporting flexible processes through recommendations based on history. In: Dumas, M., Reichert, M., Shan, M.-C. (eds.) *BPM 2008. LNCS*, vol. 5240, pp. 51–66. Springer, Heidelberg (2008)
15. Witten, I.H., Frank, E.: *Data Mining: Practical Machine Learning Tools and Techniques*, 2nd edn. Morgan Kaufmann Publishers Inc., San Francisco (2005)